MODUL 6 · TYPESCRIPT DALAM NEXT.JS

Interface vs Type untuk Props

Dua cara memberi tipe pada props komponen

Contoh Props

tombol.tsx

```
function Tombol({ judul }: { judul: string }) {
  return <button>{judul}</button>;
}

<Tombol judul={42} />
// Error: Type 'number' is not assignable to type 'string'.
```

Props = Parameter Komponen

Komponen React adalah fungsi, dan props adalah objek yang menjadi parameternya.

1

Tanpa tipe: kesalahan lolos sampai dijalankan

Salah ketik nama prop atau mengirim angka padahal butuh teks, sama seperti risiko JavaScript di sub-bab 1

2

Dengan tipe: editor mengingatkan langsung

Prop wajib yang lupa diisi, prop yang tidak dikenal, dan salah tipe nilai

Membedah Satu Baris Props

function Tombol({ judul }: { judul: string }) — dua "judul" itu punya peran berbeda.

1. **Kiri: { judul } destructuring (JavaScript)**
   Mengambil properti judul dari objek props dan menjadikannya variabel judul

2. **Kanan: { judul: string } tipe (TypeScript)**
   Objek props harus punya properti judul bertipe string

3. **Titik dua di tengah: "yang kiri bertipe yang kanan"**
   Nama di kiri dan di kanan harus sama; kalau beda, TypeScript menolak

Contoh Tanpa dan Dengan Destructuring

Dua cara menulis komponen yang sama, dan apa yang terjadi kalau nama kiri-kanan tidak cocok.

destructuring.tsx

```tsx
// Tanpa destructuring
function Tombol(props: { judul: string }) {
  return <button>{props.judul}</button>;
}

// Dengan destructuring (lebih umum)
function Tombol({ judul }: { judul: string }) {
  return <button>{judul}</button>;
}

// Nama kiri dan kanan beda
function Tombol({ judul }: { title: string }) {
// Error: Property 'judul' does not exist on type '{ title: string; }'.
```

Dua Cara Menamai Tipe Props

Inline cepat, tapi sulit dibaca begitu field-nya bertambah. Beri nama pada tipenya.

1. **interface NamaProps { ... }**
   Deklarasi khusus untuk bentuk sebuah objek

2. **type NamaProps = { ... }**
   Alias yang berarti memberi nama pada sebuah tipe, termasuk bentuk objek

Perbdedaan Interface dan Type

Props yang sama, ditulis dengan dua cara. Tanda ? berarti opsional.

interface

```
interface TombolProps {
  judul: string;
  aktif?: boolean;
}
```

type

```
type TombolProps = {
  judul: string;
  aktif?: boolean;
};
```

Contoh Memakai Props yang Sudah Bertipe

Hasilnya sama, baik memakai interface maupun type.

tombol.tsx

```tsx
function Tombol({ judul, aktif }: TombolProps) {
  return <button disabled={!aktif}>{judul}</button>;
}

<Tombol judul="Simpan" />

<Tombol />
// Error: Property 'judul' is missing in type '{}'
// but required in type 'TombolProps'.

<Tombol judul="Simpan" aktif="ya" />
// Error: Type 'string' is not assignable to type 'boolean | undefined'.
```

interface vs type: Apa Bedanya?

Untuk props, hampir semua hal bisa dilakukan keduanya. Perbedaan nyata ada di sini:

	interface	type
Memperluas	extends	intersection (&)
Dideklarasikan dua kali	Digabung otomatis (declaration merging)	Error: Duplicate identifier
Bisa menamai	Hanya bentuk objek	Apa saja: objek, union, primitif, tuple, tipe fungsi
Properti bentrok saat memperluas	Error langsung	Diam-diam menjadi never

extends vs Intersection

Dua cara membuat KartuProps yang menambah deskripsi di atas DasarProps.

interface + extends

```
interface DasarProps {
  judul: string;
}

interface KartuProps
  extends DasarProps {
  deskripsi: string;
}
```

type + &

```
type DasarProps = {
  judul: string;
};

type KartuProps =
  DasarProps & {
    deskripsi: string;
  };
```

Contoh penggunaan dalam deklarasi variable

Union, tipe turunan, dan tipe hasil Zod

hanya-type.ts

```ts
type Status = "aktif" | "nonaktif";          // union

interface KartuProps { judul: string; deskripsi: string }
type Ringkas = Pick<KartuProps, "judul">;  // tipe turunan

const schema = z.object({ nama: z.string() });
type Form = z.infer<typeof schema>;          // tipe dari Zod

const s: Status = "mati";
// Error: Type '"mati"' is not assignable to type 'Status'.
```

Pola Props yang Sering Muncul

1. **Props opsional: aktif?: boolean**
   Boleh tidak diisi; nilainya jadi undefined, bisa diberi nilai bawaan saat destructuring

2. **Props fungsi (callback): onKlik: () => void**
   Tipe fungsi: tanpa parameter, tanpa nilai kembalian

3. **children: React.ReactNode**
   Isi yang ditaruh di antara tag pembuka dan penutup komponen

4. **children tidak bisa dibatasi jenis elemennya**
   Menurut dokumentasi React, TypeScript tidak bisa memaksa children hanya berupa

Tiga Pola Props dalam Satu Komponen

Opsional, callback, dan children — ditulis dalam satu interface.

kotak.tsx

```tsx
interface KotakProps {
  judul: string;
  aktif?: boolean;            // opsional
  onKlik: () => void;         // fungsi callback
  children: React.ReactNode; // isi di dalam tag
}

function Kotak({ judul, aktif = true, onKlik, children }: KotakProps) {
  // ...
}
```

Kapan Memakai yang Mana?

Pakai interface

Bentuk objek murni

- Props komponen
- Bentuk objek yang mungkin diperluas (extends)
- Menambah properti ke tipe yang sudah ada (merging)

Pakai type

Selain bentuk objek

- Union dan literal union
- Tipe turunan: Pick, Omit, typeof
- Tipe hasil Zod: z.infer

Aturan di course ini

Props objek → interface; union, turunan, dan hasil Zod → type. Panduan resmi: pakai interface sampai butuh fitur type. Yang terpenting: konsisten.